

TEORI BAHASA DAN AUTOMATA

Modul Pembelajaran: Non-Deterministic Finite Automata (NFA)

Penjelasan mendalam mengenai konsep non-determinisme, transisi epsilon (ϵ), definisi matematis, dan studi kasus perancangan mesin NFA.

Contents

1	Mengenal Non-Deterministic Finite Automata (NFA)	2
1.1	Apa itu Mesin NFA dan Bedanya dengan DFA?	2
1.2	Syarat Diterimanya String di NFA	2
2	Transisi Kosong: Epsilon (ϵ)	3
2.1	Gimana Cara Kerja Transisi Epsilon?	3
3	Definisi Formal dan Tabel Transisi NFA	4
3.1	Tabel Transisi NFA	4
4	Kumpulan Studi Kasus Perancangan NFA	5
4.1	Kasus 1: Pola Berakhiran "01"	5
4.2	Kasus 2: Mengandung Substring "11" atau "00"	5
4.3	Kasus 3: NFA Tanpa Jalan Buntu (Mirip DFA)	5

1 Mengenal Non-Deterministic Finite Automata (NFA)

1.1 Apa itu Mesin NFA dan Bedanya dengan DFA?

Kalau sebelumnya kita udah belajar DFA yang sifatnya serba pasti dan kaku banget, sekarang kita kenalan sama saudaranya yang jauh lebih fleksibel, yaitu *Non-Deterministic Finite Automaton* (NFA). Secara fisik bayangannya mirip: ada mesin, ada pita input (*Input Tape*), dan head yang ngebaca karakter satu per satu dari kiri ke kanan.

Tapi, **cara kerjanya beda banget**. Sifat "Non-Deterministik" di sini bikin mesin NFA punya kemampuan buat "menebak" atau bercabang. Di NFA, kalau mesin dapet satu input, dia nggak cuma punya satu jalan pasti. Dia bisa bereaksi dengan tiga cara ini:

- **Bercabang (Multiple Paths):** Satu karakter input bisa bikin mesin loncat ke lebih dari satu *state* sekaligus.
- **Jalan Buntu (Dead End):** Mesin bisa aja nggak punya jalur panah transisi buat suatu karakter tertentu. Kalau dapet karakter itu, cabangnya langsung mati (gagal).
- **Gerak Gaib (Epsilon Transition):** Mesin bisa loncat ke *state* lain secara tiba-tiba tanpa perlu ngebaca karakter apa pun dari pita! Ini yang paling sakti di NFA.

Analogi Sederhana NFA

Bayangin NFA ini kayak kamu lagi main game yang ada jalan bercabang. Tiap ketemu persimpangan, karakter kamu bakal "ngekloning" dirinya sendiri dan nyusurin semua jalan yang ada secara bersamaan. Kalau salah satu aja dari kloningan itu nyampe ke garis finish (*Final State*) pas waktu habis, kamu menang (*Accept*). Kloningan lain yang nyasar atau mati di jalan buntu nggak bakal ngaruh selama ada satu yang sukses.

1.2 Syarat Diterimanya String di NFA

Karena mesinnya jalan bercabang secara paralel, aturan main diterimanya string jadi lebih longgar dibandingkan DFA.

- **Diterima (Accept):** String diterima kalau ada **setidaknya satu** jalur komputasi (salah satu kloningan) yang berhasil berhenti pas di *Final State* setelah karakter terakhir dibaca.
- **Ditolak (Reject):** String cuma bakal ditolak kalau **semua** jalur yang dicoba ternyata jalan buntu, atau ujung-ujungnya berakhir di *state* biasa (bukan *Final State*).

2 Transisi Kosong: Epsilon (ϵ)

Ini adalah salah satu fitur paling unik dari NFA yang sering banget dipakai, yaitu Transisi Epsilon (ϵ -transition). Di beberapa buku teks, kadang ditulis pakai lambda (λ), tapi maksudnya sama aja.

2.1 Gimana Cara Kerja Transisi Epsilon?

Biasanya, buat pindah dari *state* A ke *state* B, mesin butuh "ongkos" berupa membaca satu karakter dari pita input. Nah, panah yang dilabeli ϵ ini artinya jalan tol gratis. Mesin boleh pindah ke *state* tujuan tanpa perlu mengonsumsi atau membaca karakter dari pita sama sekali.

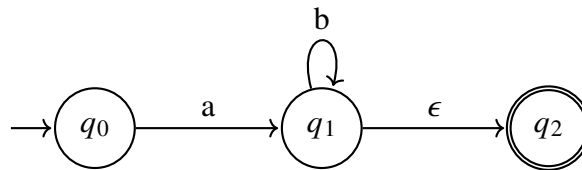


Figure 1: Contoh Transisi Epsilon. Dari q_1 , mesin bisa seketika berada di q_2 tanpa baca huruf apa pun.

Fitur ini ngebantu banget pas kita mau merancang mesin untuk mendeteksi berbagai pola bahasa yang kompleks, karena kita bisa menghubungkan berbagai mesin kecil jadi satu kesatuan dengan gampang pakai jalan tol ϵ ini.

3 Definisi Formal dan Tabel Transisi NFA

Sama kayak DFA, NFA juga digambarkan pakai model matematika 5-tupel: $M = (Q, \Sigma, \delta, q_0, F)$. Bedanya cuma ada di satu tempat, yaitu di ****Fungsi Transisinya (δ)****.

- Q : Himpunan semua *state*.
- Σ : Himpunan alfabet (karakter input). Tapi ingat, ϵ bukan bagian dari alfabet input ya!
- q_0 : *State* awal.
- F : Himpunan *Final State*.
- δ (**Fungsi Transisi**): Nah ini bedanya. Di NFA, pemetaannya adalah $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$.

Maksud dari $\mathcal{P}(Q)$ (Powerset / Himpunan Kuasa) itu apa? Artinya, hasil dari fungsi transisi NFA **bukanlah satu state tunggal**, melainkan **sebuah himpunan** state. Himpunan ini bisa isinya satu state, bisa isinya banyak state, atau bahkan bisa himpunan kosong ($\emptyset$) kalau ternyata jalannya buntu.

3.1 Tabel Transisi NFA

Karena hasil pemetaannya berupa himpunan, cara nulis di tabel transisinya juga harus pakai kurung kurawal $\{\}$.

Misal kita punya NFA di mana di state q_0 baca '0' bisa ke q_0 atau ke q_1 , baca '1' cuma ke q_0 , dan ada transisi ϵ dari q_1 ke q_2 .

δ	0	1	ϵ
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$	$\emptyset$
q_1	$\emptyset$	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$	$\emptyset$

Table 1: Tabel Transisi NFA. Perhatikan pemakaian kurung kurawal dan himpunan kosong ($\emptyset$).

4 Kumpulan Studi Kasus Perancangan NFA

Merancang NFA itu rasanya jauh lebih gampang dan lebih natural dibanding ngerancang DFA. Kenapa? Karena kita tinggal fokus bikin jalur buat string yang benar aja. Kalau buat string yang salah, gampang, biarin aja buntu!

4.1 Kasus 1: Pola Berakhiran "01"

Kita pengen bikin mesin buat bahasa $L = \{w \mid w \text{ diakhiri dengan } 01\}$. Karakter apa pun di depannya bebas.

Kalau pakai DFA, ini bakal lumayan mikir karena harus bikin *loop back* kalau polanya gagal di tengah jalan. Tapi kalau pakai NFA, desainnya *straightforward* banget!

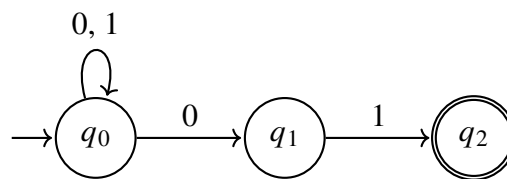


Figure 2: NFA String Berakhiran "01"

Bedah Alurnya (Misal input "1001"):

- Awal di q_0 . Baca '1', cuma bisa ngeloop di q_0 .
- Baca '0' pertama: mesin ngekloning diri. Satu tetap di q_0 , satu lagi maju ke q_1 .
- Baca '0' kedua: kloningan di q_0 ngekloning diri lagi (ada yang tetep di q_0 , ada yang maju ke q_1). Kloningan yang tadi di q_1 malah **mati/buntu** karena di q_1 nggak ada panah buat huruf '0'.
- Baca '1' terakhir: kloningan yang ada di q_1 melesat ke q_2 (*Final State*). Kloningan lain biarin aja terserah di mana. Karena ada satu yang nyampe di q_2 , maka **Diterima**.

4.2 Kasus 2: Mengandung Substring "11" atau "00"

Gimana kalau kita mau deteksi string yang ada "11" berurutan ATAU "00" berurutan di dalamnya? Di sini transisi ϵ bakal sangat bersinar.

Kita pecah jadi tiga bagian: 1. Mesin tebakan awal (start). 2. Mesin pendeteksi "11". 3. Mesin pendeteksi "00".

Kenapa ini gampang banget dibikin? Pas mesin dinyalain, berkat transisi ϵ , mesin seketika itu juga ngekloning dirinya sendiri tanpa baca apa-apa. Satu kloningan nangkring di q_1 buat nungguin dan nyari pola "11", sementara kloningan satunya lagi nangkring di p_1 buat nyari pola "00". Mereka kerja barengan secara paralel. Kalau salah satu dari pola itu ketemu duluan, beres! String langsung diterima.

4.3 Kasus 3: NFA Tanpa Jalan Buntu (Mirip DFA)

Perlu diingat, setiap DFA itu adalah NFA, tapi nggak semua NFA itu DFA. NFA yang nggak pakai fitur *multiple path* atau epsilon sama sekali, ya sebenarnya bentuknya persis kayak DFA.

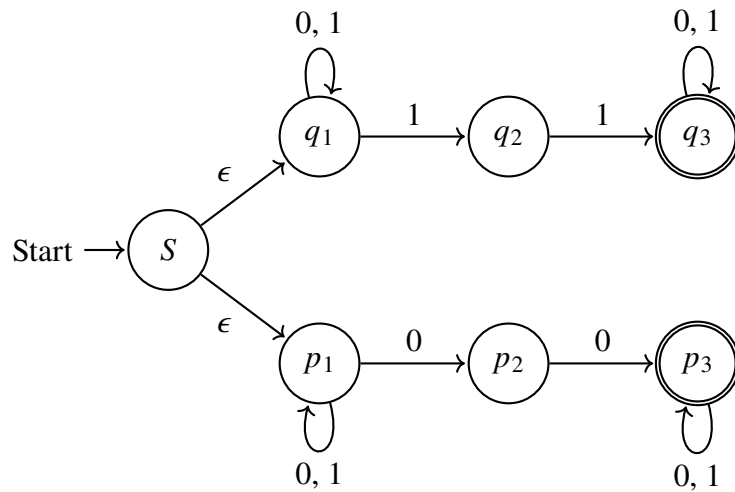


Figure 3: NFA dengan Transisi Epsilon mendeteksi "11" atau "00"

NFA cuma ngasih kita kebebasan: "Kamu boleh kok nggak nulis semua panahnya kalau emang nggak butuh." Kebebasan inilah yang bikin NFA jadi andalan pas kita mau ngedesain pola bahasa untuk tahap awal, sebelum nanti di- *convert* jadi DFA buat urusan **coding** di program.